ResourceFlow

Intelligent Meeting Room & Banquet Scheduling Platform

Project Specification & Engineering Rationale

This document explains, in full, what this project is, why it exists, the exact problem it solves, how each part of the system works, the technical decisions behind it, and why it is a strong, defensible project for interviews and placements.

1. What We Are Building

ResourceFlow is a **multi-tenant scheduling platform** that helps organizations manage shared meeting rooms and banquet/community halls. Any organization — a housing society, a startup, a coworking space, or a college — can sign up, list its rooms/halls, and let its members book them through the platform instead of coordinating manually.

The MVP is intentionally scoped to two resource types only: **meeting rooms** and **banquet/community halls**. These were chosen deliberately because they have the richest scheduling constraints (capacity, equipment, location, approval needs) — making the scheduling engine genuinely interesting to build, while the architecture remains extensible to vehicles, cameras, or sports equipment later.

2. The Problem We Are Solving

Small organizations today manage shared rooms and halls using WhatsApp groups, spreadsheets, or verbal coordination. This consistently leads to:

- **Double bookings** — two people believe they have the same room at the same time.
- **Manual back-and-forth coordination** — someone has to manually check and confirm availability.
- **Poor utilization visibility** — no one knows which rooms are overused or sitting idle.
- **Last-minute confusion** — conflicts are discovered only when people physically show up.
- **No structured approval process** — important bookings and casual ones are treated identically.

None of this requires a complicated app to fix — but fixing it *well*, in a way that scales to concurrent users and handles real edge cases, is a legitimate engineering problem. That is the gap this project fills.

3. Core Design Philosophy

> ***"Suggest. Never decide."***

The platform never automatically moves, cancels, or reassigns anyone's booking. Whenever a conflict is detected, it presents ranked, explainable alternatives — and a human (the member or the admin) always makes the final decision. This is deliberate: automatic reassignment is unpredictable, hard to justify to upset users, and unnecessary complexity for a project of this scope. Keeping a human in the loop keeps the system simple to reason about and mirrors how real approval-based business systems are actually built.

4. How the System Solves the Problem

4.1 Conflict-Free Booking

Every booking request is checked against existing bookings for that room. If the requested time range does not overlap with any existing approved booking, the request proceeds. If it does overlap, the booking is rejected outright — and the recommendation engine takes over to offer alternatives.

4.2 The Recommendation Engine (the heart of the project)

Instead of simply saying *"Unavailable"*, the system finds and ranks real alternatives. This happens in two steps:

Step 1 — Hard Filtering (Rule Engine)

Every candidate room must satisfy mandatory constraints before it is even considered:

- Belongs to the same organization (multi-tenant isolation)
- Is actually available for the requested (or a nearby) time window
- Has capacity ≥ the requested capacity
- Has the required equipment (projector, whiteboard, etc.), if specified

Step 2 — Weighted Scoring (Ranking)

Rooms that pass the filter are then scored using a transparent, explainable formula — not a black-box model:

Factor	Rule	Max Score
Capacity fit	25 – min(25, |room.capacity – requested.capacity| × 2)	25
Equipment match	Flat score if all required equipment is present	20
Same floor	Bonus if room is on the same floor as requested room	15
Same building	Bonus if room is in the same building	10
Utilization balance	Higher score for less-used rooms (load balancing)	7

The top-scoring rooms (e.g. top 5) are shown to the user as alternatives, each with its score and the reason it qualified. The user picks one, if any, and submits it as a new booking request. Nothing is booked automatically.

Why rule-based instead of Machine Learning?

An earlier version of this design considered an ML-based ranking layer (e.g. XGBoost) trained on historical booking-acceptance data. This was deliberately removed for the MVP: a new platform with no real users has no genuine historical data to train on, and fabricating synthetic data to justify an ML feature is a weak position to defend in an interview. A deterministic, weighted rule engine is explainable, debuggable, and requires no training data — and the design leaves room to swap in a learned ranking model later, once real usage data exists.

4.3 Two Points Where Suggestions Appear

A. Request-time suggestions — when a member requests a room that turns out to be unavailable, the engine immediately shows ranked alternatives (different room, different time slot, different day).

B. Approval-time suggestions — when an admin is reviewing a higher-priority booking that conflicts with an existing one, the system suggests possible resolutions (move the existing booking to Room B, offer another slot). The admin decides whether to act on it — nothing changes automatically.

4.4 Booking Lifecycle (State Machine)

Every booking moves through clearly defined states rather than a simple true/false "booked" flag:

Requested → **Approved** → **Active** → **Completed**

or, alternatively: Requested → **Rejected**

This mirrors how real-world systems (order processing, ticketing platforms) manage state, and makes the approval workflow explicit rather than implicit.

4.5 Preventing Double-Booking Under Concurrency

The hardest real bug in any booking system: two users click "Book" on the same room at nearly the same instant. If both requests check availability before either one writes to the database, both can succeed — a classic race condition.

This is solved not in application code, but at the database level, using a **PostgreSQL exclusion constraint** (`EXCLUDE USING gist`) on `(room_id, time_range)`. This makes it structurally impossible for two overlapping bookings to exist for the same room, no matter how many requests arrive simultaneously or how many backend servers are running. If an insert violates the constraint, the database itself rejects it, and the application catches that rejection and triggers the recommendation engine.

4.6 Multi-Tenant Architecture

Every organization (a housing society, a company, a college club) operates in a fully isolated workspace. Members of one organization can never see or book resources belonging to another, even though everyone shares the same underlying platform — the same core principle behind real SaaS products.

4.7 Role-Based Access Control

Role	Capabilities
Admin	Add/edit/remove rooms, approve or reject bookings, manage members, view analytics

Member	Browse rooms, request bookings, accept suggested alternatives, view booking history

4.8 Analytics Dashboard

Admins get a simple, well-defined set of metrics to help decide whether they actually need more (or fewer) rooms:

- **Utilization %** = (Booked Hours ÷ Available Hours) × 100, over a chosen period
- Most frequently booked room
- Peak booking hours
- Frequently rejected requests (signals under-supply of a resource)

Example: a room available 9am–5pm (8 hrs/day) that is booked for 5 hours has 62.5% utilization that day. Over a 5-day week (40 available hours) with 26 booked hours, utilization is 65%. This exact, reproducible formula is what makes the metric defensible rather than a vague dashboard number.

5. Database Design (PostgreSQL)

Core tables:

Table	Purpose
organizations	One row per tenant (society, company, club)
users	Belongs to an organization; has a role (admin/member)
rooms	Capacity, building, floor, equipment, available hours
bookings	Room, user, start/end time, status, priority
booking_status_log	History of state transitions for audit/analytics

Key indexes: `organization_id`, `room_id`, and a composite index on `(room_id, start_time, end_time)` to make overlap checks fast even as booking volume grows. The overlap-prevention rule itself lives in an exclusion constraint on the bookings table, not just an index.

6. Technology Stack

Layer	Technology
Frontend	Next.js + Tailwind CSS
Backend	Express.js
Database	PostgreSQL
ORM	Prisma
Authentication	JWT
Deployment	Docker, Vercel (frontend), Railway/Render (backend + DB)

7. What Was Deliberately Cut From the MVP (and why)

Feature	Reason for Cutting
ML-based ranking	No real historical data exists yet to honestly train or defend a model
Dynamic pricing	Doesn't fit the actual use case (societies/clubs won't surge-price a hall)
Split bookings (e.g. 6hrs across 2 rooms)	Hides many edge cases (partial cancellation, cost split, physically moving) — better as a v2 feature
Vehicles / cameras / sports equipment	MVP intentionally narrowed to 2 resource types with the richest constraints; architecture supports ad
QR check-in, waitlists, calendar sync, notifications	Useful, but not core to proving the scheduling engine works

Explicitly cutting these and stating the reasoning is itself a good signal in an interview: it shows deliberate scoping rather than an inability to build them.

8. Build Timeline (4–5 Weeks)

Week	Focus
Week 1	Auth (JWT), organization management, room CRUD, PostgreSQL schema
Week 2	Booking engine, conflict detection, exclusion constraint, state machine
Week 3	Recommendation engine: filtering + weighted scoring, approval workflow
Week 4	Analytics dashboard, testing, UI polish
Week 5 (buffer)	Deployment, documentation, README, demo video/script

9. Why This Is a Strong Resume Project

This is not "another booking app." The engineering value is in the scheduling and conflict-resolution logic underneath a simple-looking UI. Concretely, it gives you real, defensible talking points on:

- Interval-overlap conflict detection
- Preventing race conditions using database-level constraints, not application code
- A weighted, explainable recommendation/ranking algorithm
- A booking lifecycle modeled as a finite state machine
- Multi-tenant data isolation
- Role-based authorization
- Relational schema design and indexing for a real, non-trivial domain

These map directly to the kinds of questions asked in SDE interviews: concurrency handling, database transactions, system design tradeoffs, and algorithmic reasoning — not just "which frontend library did you use."

10. The One-Line Pitch

> *"I built a multi-tenant scheduling platform for meeting rooms and banquet halls that prevents double-bookings at the database level using PostgreSQL exclusion constraints, and resolves conflicts with an explainable, rule-based recommendation engine instead of ML — since ML ranking couldn't be honestly justified without real usage data."*
